Search & Navigation Challenge

ROS2 Package — Software Design Document

COSC2781 (PG) / COSC2814 (UG) · Semester 1 2026 · v1.0

1. Overview

This document describes the software design of the **snc_robot** ROS2 package developed for the Search & Navigation Challenge (SNC). The package must enable a ROSBot Pro to autonomously explore an unknown closed space, locate five hazard markers, publish their map-frame positions, and return to its starting location — all within a seven-minute window.

The implementation is divided into three custom ROS2 nodes, a shared constants module, a central launch file, and a tunable parameter configuration file. All nodes are authored in Python using **rclpy**, the ROS2 Python client library.

1.1 Goals

- **Correctness:** publish all five required output topics with valid, map-frame data.
- **Speed:** complete the challenge well within the 7-minute limit.
- **Robustness:** handle contingencies (teleop fallback, manual triggers) without crashing.
- **Maintainability:** single shared constants module; tunable parameters in one YAML file.

1.2 Scope

This document covers software architecture, inter-node communication design, algorithm selection rationale, the configuration file schema, and the complete parameter reference. Hardware bringup (ROSBot firmware, USB connections) and TheConstruct simulation setup are out of scope.

2. System Architecture

2.1 Package structure

```
snc_robot/
  snc/
    __init__.py
    constants.py            # shared IDs, topic names, status strings
    node1_navigation.py     # Node 1 — exploration logic
    node2_detection.py      # Node 2 — hazard detection & localisation
    node3_pathing.py        # Node 3 — path tracking & return home
  launch/
    snc.launch.py           # top-level launch: all nodes + SLAM + Nav2 +
find_object_2d
  config/
    params.yaml             # all tunable parameters (see Section 5)
    nav2_params.yaml        # Nav2 costmap / planner config
    mapper_params.yaml      # SLAM Toolbox online-async config
  resource/
    markers/                # find_object_2d training images (13 hazard + start)
  evidence/
    index.md                # index of all video/bag evidence files
  package.xml
  setup.py
  README.md
```

2.2 Node overview

Node	File	Primary role	Owner
Node 1	`node1_navigation.py`	Frontier exploration, Nav2 goal dispatch, spin-on-detect, teleop fallback	Student A
Node 2	`node2_detection.py`	Start-card & hazard detection, depth-based localisation, /hazards publisher	Student B
Node 3	`node3_pathing.py`	Path recording, waypoint reversal, return-home navigation, path publishers	Student C

2.3 External packages

Package	Purpose
`slam_toolbox`	Online-async SLAM — builds and publishes /map, provides map→odom TF
`nav2_bringup`	NavigateToPose and FollowWaypoints action servers, obstacle-aware planning
`find_object_2d`	Template-matching object detector; outputs object ID + homography on /objects
`tf2_ros`	Transform lookups: map→base_link, map→camera_frame, map→laser_frame
`nav_msgs`	OccupancyGrid (/map), Path (/path_explore, /path_return)
`visualization_msgs`	Marker messages for /hazards

`sensor_msgs`	LaserScan (/scan), Image + CameraInfo (/camera/*)
`std_msgs`	/snc_status (String), trigger topics (Empty)

3. Inter-Node Communication

3.1 Required output topics (marking script)

Topic	Message type	Publisher	frame_id
`/map`	`nav_msgs/OccupancyGrid`	SLAM Toolbox	`map`
`/hazards`	`visualization_msgs/Marker`	Node 2	`map`
`/path_explore`	`nav_msgs/Path`	Node 3	`map`
`/path_return`	`nav_msgs/Path`	Node 3	`map`
`/snc_status`	`std_msgs/String`	All nodes	—

3.2 Internal inter-node topics (custom)

Three lightweight custom topics coordinate the nodes without tight coupling. All names are defined in constants.py so they can be changed in one place.

Topic	Type	Direction	Purpose
`/snc_start`	`std_msgs/Empty`	Node 2 → Node 1	Node 2 fires this when the start card is detected (or /trigger_start received). Node 1 begins exploration on receipt.
`/snc_hazard_signal`	`std_msgs/Empty`	Node 2 → Node 1	Fired by Node 2 each time a marker is confirmed. Node 1 cancels its current Nav2 goal and spins 360°.
`/snc_return_trigger`	`std_msgs/Empty`	Node 1 → Node 3	Fired by Node 1 when all 5 markers are found or the 4-minute timeout elapses. Activates Node 3 return-home.

3.3 Contingency / assessor trigger topics

Topic	Type	Behaviour on receipt
`/trigger_start`	`std_msgs/Empty`	Equivalent to visual start-card detection — Node 2 publishes /snc_start.
`/trigger_teleop`	`std_msgs/Empty`	Node 1 cancels all Nav2 goals and enters passive mode; teleop node may take over /cmd_vel.
`/trigger_home`	`std_msgs/Empty`	Node 3 begins return-home immediately, regardless of whether exploration ended normally.

4. Node Designs

4.1 Node 1 — Navigation Logic

Algorithm: frontier-based exploration

Node 1 uses **frontier-based exploration** as its primary strategy. On a timer it reads the occupancy grid from `/map`, identifies **frontier cells** (cells marked as free that are adjacent to at least one unknown cell), clusters them spatially, and dispatches a **NavigateToPose** action goal toward the nearest cluster centroid that has not recently been attempted. This guarantees coverage of the entire reachable space without requiring any pre-built map.

Custom logic added on top of the stock frontier algorithm:

- **Spin-on-detect:** when /snc_hazard_signal is received, the current Nav2 goal is cancelled and the robot issues a 360° spin via /cmd_vel before selecting the next frontier. This maximises the camera's sweep angle around a detected marker.
- **Marker-count guard:** the node monitors the count of unique IDs published on /hazards. Once 5 unique IDs are seen, it fires /snc_return_trigger without waiting for the timeout.
- **Timeout guard:** a 4-minute wall-clock timer fires /snc_return_trigger if markers are not all found in time.
- **Teleop fallback:** on receipt of /trigger_teleop, all Nav2 goals are cancelled and the node publishes STATUS_TELEOP; it resumes listening for /snc_start if re-triggered.

State machine

State	Trigger in	Action
WAITING	/snc_start received	Transition → EXPLORING; start frontier timer
EXPLORING	Nav2 goal succeeded/failed	Select next frontier centroid; dispatch new goal
SPINNING	/snc_hazard_signal received	Cancel goal; spin 360°; transition → EXPLORING
RETURNING	/snc_return_trigger received	Cancel goal; notify Node 3; update status
TELEOP	/trigger_teleop received	Cancel goal; suspend; wait for manual driving

Key subscriptions and publications — Node 1

Topic	Direction	Why
`/map`	Subscribe	Read OccupancyGrid to identify frontiers
`/snc_start`	Subscribe	Begin exploration
`/snc_hazard_signal`	Subscribe	Trigger spin behaviour
`/trigger_teleop`	Subscribe	Enter passive/teleop mode
`/hazards`	Subscribe	Count unique confirmed markers
`/cmd_vel`	Publish	Spin commands during spin-on-detect
`/snc_return_trigger`	Publish	Signal Node 3 to begin return-home

`/snc_status`	Publish	Human-readable state string for assessor

4.2 Node 2 — Hazard Detection & Localisation

Detection pipeline

Node 2 subscribes to `/objects` from **find_object_2d**, which runs template matching against the 14 training images (13 hazard markers + start card). Each detection message carries an object ID and a 2D homography matrix describing the bounding box in image coordinates.

The detection pipeline applies three filters before acting on a detection:

- **Confidence filter:** the bounding-box area (derived from the homography) must exceed marker_min_bbox_area (params.yaml) to reject distant or partial detections.
- **Duplicate guard:** a set of already-confirmed marker IDs is maintained. If a detection's SNC hazard ID is already in this set, it is silently dropped — the robot will not re-localise a marker it has already published.
- **Start-card routing:** if the detected ID matches START_MARKER_OBJECT_ID (constants.py), the detection is routed to the start-trigger publisher, never to the hazard localisation pipeline.

Localisation method: depth image back-projection

On a validated detection, Node 2 extracts the **pixel coordinates of the bounding-box centre** from the homography. It then:

- Reads the depth value at (cx, cy) from /camera/depth/image_raw.
- Back-projects to a 3D point in camera frame using the camera intrinsics from `/camera/depth/camera_info`: X = (cx − ppx) × depth / fx, Y = (cy − ppy) × depth / fy, Z = depth.
- Transforms the 3D point from camera_frame → map using `tf2_ros.Buffer.lookup_transform('map', 'camera_depth_optical_frame', rclpy.time.Time())`
- **Median position filter:** readings are buffered over marker_localisation_window_s seconds. The median (x, y) is published once the buffer contains at least marker_localisation_min_samples readings. This prevents a single noisy depth reading from producing a wildly incorrect marker placement.

Hazard marker ID mapping

The mapping between find_object_2d training image filenames and SNC marker IDs is defined exclusively in constants.py:

```
HAZARD_ID_MAP = {
    1:  'explosive',            2:  'flammable_gas',
    3:  'non_flammable_gas',    4:  'dangerous_when_wet',
    5:  'flammable_solid',      6:  'spontaneously_combustible',
    7:  'oxidizer',             8:  'organic_peroxide',
    9:  'inhalation_hazard',    10: 'poison',
    11: 'radioactive',          12: 'corrosive',
}
START_MARKER_SNC_ID = 0   # never published to /hazards
```

Key subscriptions and publications — Node 2

Topic	Direction	Why
`/objects`	Subscribe	find_object_2d detections (ID + homography)
`/camera/depth/image_raw`	Subscribe	Depth values for back-projection

`/camera/depth/camera_info`	Subscribe	Intrinsics (fx, fy, ppx, ppy)
`/trigger_start`	Subscribe	Assessor contingency — route to /snc_start
`/snc_start`	Publish	Signal Node 1 to begin exploration
`/hazards`	Publish	Confirmed marker positions in map frame
`/snc_hazard_signal`	Publish	Notify Node 1 to spin after each new marker
`/snc_status`	Publish	State string including detected marker name

4.3 Node 3 — Path Tracking & Return Home

Path recording

Node 3 runs a timer at `1 / pose_sample_interval_s` Hz. On each tick it calls `tf2_ros.Buffer.lookup_transform('map', 'base_link', rclpy.time.Time())` to get the current map-frame pose. A new pose is only appended to the exploration path list if **either** of the following thresholds are exceeded since the last recorded pose:

- **Distance:** `Δposition ≥ waypoint_spacing_m` (default 0.15 m)
- **Rotation:** `Δyaw ≥ waypoint_min_rotation_deg` (default 15°)

This dual-threshold strategy keeps path density suitable for Nav2 waypoint following — neither too dense (causing goal thrashing) nor too sparse (causing the return path to miss corridors).

The growing path is published on `/path_explore` after each append. Recording stops when `/snc_return_trigger` is received.

Return-home path construction

On return trigger, Node 3 constructs the return waypoint list as follows:

- Reverse the exploration pose list: `return_poses = list(reversed(explore_poses))`
- **Invert quaternions:** for each pose, the orientation is recomputed to face the direction of travel. Specifically, the heading is derived from the vector between consecutive waypoints (`atan2(Δy, Δx)`), and a fresh quaternion is constructed from this yaw angle. This ensures the robot faces forward rather than reversing.
- **Thin the list:** if consecutive reversed poses are closer than `waypoint_spacing_m`, intermediate poses are skipped until the spacing threshold is met. This removes redundancy introduced by the robot slowing near previously sent goals.

Return navigation strategy

The thinned reversed waypoint list is sent to Nav2's **FollowWaypoints** action server (`nav2_msgs/action/FollowWaypoints`). This sends all waypoints in a single action call and lets Nav2 handle the sequencing, obstacle avoidance around the pre-recorded path, and recovery if a waypoint becomes unreachable.

The robot halts and publishes zero velocity once the home position (first recorded exploration pose) is within home_arrival_radius_m. It holds that position for ≥ 5 seconds before publishing STATUS_COMPLETE.

Key subscriptions and publications — Node 3

Topic	Direction	Why
`/snc_return_trigger`	Subscribe	Begin return-home and stop exploration recording
`/trigger_home`	Subscribe	Assessor manual contingency for return-home
`/path_explore`	Publish	Growing exploration path in map frame
`/path_return`	Publish	Growing return path in map frame
`/snc_status`	Publish	STATUS_RETURNING / STATUS_COMPLETE

5. Configuration File Reference

All tunable parameters live in `config/params.yaml`. Each node loads its section at startup via the ROS2 parameter server. Values shown are the recommended defaults; they should be tuned empirically during real-robot testing.

```yaml
# ────────────────────────────────────────────────────────────────────────
# config/params.yaml  —  SNC ROS2 package tunable parameters
# Loaded by snc.launch.py; all nodes read from the /snc_robot namespace.
# ────────────────────────────────────────────────────────────────────────

snc_robot:

  # ── Node 1: Navigation ──────────────────────────────────────────────
  node1:

    # How long the robot spins in place after a hazard signal (seconds).
    # Increase if markers are missed during spin; decrease for speed.
    spin_duration_s: 4.0

    # Spin angular velocity while scanning for markers (rad/s).
    spin_velocity_rad_s: 0.5

    # Frontier exploration timer period (seconds).
    # Shorter = more reactive; longer = less CPU load.
    frontier_update_interval_s: 2.0

    # Minimum cluster size (cells) to treat a frontier as a valid target.
    # Increase to ignore small unexplored pockets near walls.
    frontier_min_cluster_size: 5

    # Radius around a previously attempted frontier centroid (m) to mark
    # as 'visited' and skip on the next selection pass.
    frontier_visited_radius_m: 0.4

    # Wall-clock limit for exploration before forcing return-home (seconds).
    # Must be < 240 to leave time for the return journey.
    exploration_timeout_s: 220.0

    # Distance from home position considered 'arrived' during return (m).
    home_arrival_radius_m: 0.30

  # ── Node 2: Hazard Detection ────────────────────────────────────────
  node2:

    # find_object_2d object ID assigned to the start card image.
    # Must match the order images were loaded into find_object_2d.
    start_marker_object_id: 14

    # Minimum bounding-box pixel area (from homography) to accept a
    # detection as valid. Rejects distant or partially visible markers.
    marker_min_bbox_area: 4000

    # Time window over which depth readings are buffered before the
    # median position is published to /hazards (seconds).
    marker_localisation_window_s: 1.5

    # Minimum number of depth samples required in the window before
    # publishing.  Prevents premature publication from a single reading.
    marker_localisation_min_samples: 5

    # Depth values outside this range are discarded as sensor noise (m).
    depth_min_m: 0.20
    depth_max_m: 4.00
```

```
# ── Node 3: Path Tracking & Return ──────────────────────────────
node3:

  # Timer period for TF pose sampling during exploration (seconds).
  pose_sample_interval_s: 0.20

  # Minimum translational distance between consecutive recorded poses (m).
  # Combined with waypoint_min_rotation_deg: a pose is recorded when
  # EITHER threshold is exceeded.
  waypoint_spacing_m: 0.15

  # Minimum rotational change between consecutive recorded poses (degrees).
  waypoint_min_rotation_deg: 15.0

  # How long the robot holds its home position before the challenge
  # is declared complete (seconds).  Spec requires >= 5.
  home_hold_duration_s: 6.0
```

5.1 Parameter tuning guidance

Parameter	Tune if…	Effect of increasing
`spin_duration_s`	Markers missed during spin	More camera coverage; slower exploration
`frontier_min_cluster_size`	Robot wastes time on tiny pockets	Ignores small frontiers; may leave corners unexplored
`marker_min_bbox_area`	False positives at distance	More selective; may miss markers at long range
`marker_localisation_window_s`	Published positions are noisy	More stable position; slight delay before publishing
`marker_localisation_min_samples`	Single-frame outliers accepted	More robust; robot must dwell longer near marker
`waypoint_spacing_m`	Nav2 goal thrashing on return	Sparser path; return may deviate from explore route
`home_arrival_radius_m`	Robot stops too far from home	More tolerant arrival; robot may stop slightly off-centre

6. Shared Constants Module

All magic strings, topic names, and ID mappings are centralised in `snc/constants.py`. No node may hard-code a topic name or status string — it must import from this module.

```
# snc/constants.py

# ── Internal topic names ──────────────────────────────────────
TOPIC_START          = '/snc_start'
TOPIC_HAZARD_SIGNAL  = '/snc_hazard_signal'
TOPIC_RETURN_TRIGGER = '/snc_return_trigger'

# ── Required output topics ────────────────────────────────────
TOPIC_HAZARDS        = '/hazards'
TOPIC_PATH_EXPLORE   = '/path_explore'
TOPIC_PATH_RETURN    = '/path_return'
TOPIC_STATUS         = '/snc_status'

# ── Contingency trigger topics ────────────────────────────────
TOPIC_TRIGGER_START  = '/trigger_start'
TOPIC_TRIGGER_TELEOP = '/trigger_teleop'
TOPIC_TRIGGER_HOME   = '/trigger_home'

# ── Status strings ────────────────────────────────────────────
STATUS_WAITING   = 'waiting for start'
STATUS_EXPLORING = 'exploring'
STATUS_SPINNING  = 'spinning to locate marker'
STATUS_RETURNING = 'returning home'
STATUS_COMPLETE  = 'challenge complete'
STATUS_TELEOP    = 'teleop active'

def status_marker(name: str) -> str:
    return f'marker detected: {name}'

# ── Hazard marker ID map (SNC spec table) ─────────────────────
# Key = SNC hazard ID, Value = human-readable name
HAZARD_ID_MAP = {
    0:  'unknown',
    1:  'explosive',
    2:  'flammable_gas',
    3:  'non_flammable_gas',
    4:  'dangerous_when_wet',
    5:  'flammable_solid',
    6:  'spontaneously_combustible',
    7:  'oxidizer',
    8:  'organic_peroxide',
    9:  'inhalation_hazard',
    10: 'poison',
    11: 'radioactive',
    12: 'corrosive',
}

# ── Start marker ──────────────────────────────────────────────
# find_object_2d object ID for the start card (update after loading images)
START_MARKER_OBJECT_ID = 14
# The start marker must NEVER be published to /hazards
```

7. Launch File Outline

The single entry point is `launch/snc.launch.py`. It starts all processes in dependency order: SLAM first (to establish the TF tree), then Nav2 (which needs `/map`), then `find_object_2d` (needs camera), and finally the three SNC nodes.

```
# launch/snc.launch.py  —  simplified outline

def generate_launch_description():
    params = os.path.join(pkg_dir, 'config', 'params.yaml')

    slam = IncludeLaunchDescription(            # 1. SLAM Toolbox
        PythonLaunchDescriptionSource([...]),
        launch_arguments={'params_file':
            os.path.join(pkg_dir, 'config', 'mapper_params.yaml')}.items())

    nav2 = IncludeLaunchDescription(            # 2. Nav2 bringup
        PythonLaunchDescriptionSource([...]),
        launch_arguments={'params_file':
            os.path.join(pkg_dir, 'config', 'nav2_params.yaml')}.items())

    find_obj = Node(                            # 3. find_object_2d
        package='find_object_2d', executable='find_object_2d_node',
        parameters=[{'objects_path':
            os.path.join(pkg_dir, 'resource', 'markers')}])

    node1 = Node(package='snc_robot',           # 4. Navigation logic
        executable='node1_navigation',
        parameters=[params])

    node2 = Node(package='snc_robot',           # 5. Hazard detection
        executable='node2_detection',
        parameters=[params])

    node3 = Node(package='snc_robot',           # 6. Path tracking
        executable='node3_pathing',
        parameters=[params])

    return LaunchDescription([slam, nav2, find_obj, node1, node2, node3])
```

8. Risk & Contingency

Risk	Likelihood	Mitigation
Start card not detected visually	Medium	Assessor uses /trigger_start; Node 2 routes it identically to visual detection
Nav2 goal fails / robot gets stuck	Medium	Nav2 recovery behaviours (spin, back-up) activate automatically; frontier algorithm selects a new target
Marker depth reading is noisy/invalid	Medium	Depth range clamp + median filter over window; requires min_samples before publishing
Exploration misses a corner	Low	Frontier cluster size threshold tuned to pursue small unexplored regions; spin-on-detect broadens sweep
Return path deviates significantly	Low	Waypoint spacing tuned empirically; FollowWaypoints uses Nav2 local planner to stay on track
TF lookup failure / extrapolation	Low	Lookup wrapped in try/except; node logs and skips the frame rather than crashing
find_object_2d false positive	Low	Bounding-box area threshold + duplicate guard prevent erroneous /hazards publications

9. Testing Plan

9.1 Simulation (TheConstruct / Gazebo)

- Build a Gazebo world with four walls, at least two static obstacles, and five hazard marker textures placed at ≥ 10 cm height.
- Run `ros2 bag record /map /hazards /path_explore /path_return /snc_status /snc_hazard_signal` for the full simulated run.
- Inspect the bag in RViz: verify frame_id = 'map', marker IDs match spec, paths cover the explored area.
- Verify Node 1 spins visibly in RViz each time /snc_hazard_signal fires.
- Verify /path_return traces the reverse of /path_explore.

9.2 Real-robot integration tests

- Test each node in isolation before running all three together.
- Measure hazard marker placement error: physically measure marker position; compare to published /hazards coordinate. Document delta in evidence/index.md.
- Tune `waypoint_spacing_m`: run return-home with dense spacing (0.05 m) then sparse (0.30 m); choose the value that most faithfully retraces the path without Nav2 goal thrashing.
- Test all three contingency triggers using the assessor topic commands before demo day.
- Complete at least two full-challenge runs on the real robot and record video of each.